Filter FabJS Programming Guide

Formula language reference, renderer compatibility, and practical filter tutorials

For Filter FabJS v2.4.6

August 2026

Inspired by the structure of the 1997 Filter Factory Programming Guide, but rewritten for Filter FabJS's current browser-native formula engine.

Contents

0. About this guide

This guide documents the formula language implemented by Filter FabJS v2.4.6. It is written as a programming reference and tutorial rather than as an architectural specification. The goal is to make it possible to author filters directly in the four RGBA formula fields, understand what each expression means, and predict when a formula can run on WebGPU or must fall back to the CPU Worker.

The supplied 1997 Filter Factory Programming Guide is used as a structural reference: fundamentals first, then variables/operators/functions, followed by progressively richer image-processing tutorials. Filter FabJS is not a drop-in clone of the original Filter Factory language. It preserves many familiar ideas while adding native floating-point math, bilinear sampling, deterministic procedural noise, gradients, patterns, analytic masks, blend helpers, renderer selection, and stricter parser limits.

Every formula example in this guide was checked against the current Filter FabJS grammar for valid identifiers, function arity, parentheses, ternary syntax, and operator structure. Examples intentionally avoid unsupported statement syntax such as assignments, semicolons, loops, and user-defined functions.

1. What is Filter FabJS?

Filter FabJS is a browser-native procedural RGBA image-filter editor. A filter is defined by four expressions: one each for red, green, blue, and alpha. Each expression is evaluated for every output pixel. The expression can read the source pixel, sample other coordinates, use controls, generate procedural values, or combine these techniques.

The editor compiles parsed formulas into a typed intermediate representation (IR). In Auto mode, deterministic stateless formulas are sent to WebGPU when supported. Unsupported or legacy formulas are evaluated by a CPU Worker. The same filter can therefore use a compact expression language while the renderer is selected independently.

1.1 The four-channel model

The four formula fields produce the output channels R, G, B, and A. A formula may read any source channel regardless of which output field it is in. The final channel result is clamped to the displayable 0-255 range when written to the output image.

Pass-through filter

```
R: r
G: g
B: b
A: a
```

The current source pixel is always available through r, g, b, and a. The variable c means the current source channel: in the R field c is r, in the G field c is g, and so on.

1.2 Rendering model

- Auto: uses WebGPU when the parsed program is compatible and WebGPU is available; otherwise uses the CPU Worker.
- GPU: requests the WebGPU backend. Stateful or legacy-only language features are rejected by compatibility analysis.

- CPU: evaluates the typed IR in a Web Worker and supports the full current language, including legacy/stateful constructs.

WebGPU compatibility is decided from the typed IR, not from a hand-maintained preset flag. This matters when you edit a built-in filter: a small syntax change can move a formula from the GPU-compatible subset to CPU-only.

2. Programming the Formula Language

2.1 Pixel and channel model

Coordinates use a conventional raster image system. x increases from left to right and y increases from top to bottom. The top-left pixel is approximately (0,0); the last valid integer pixel coordinate is (X-1,Y-1). X and Y are the image width and height.

Symbol	Meaning	Typical range
r, g, b, a	Source red, green, blue, alpha	0..255
c	Source value for the current output channel	0..255
x, y	Current pixel coordinates	0..X-1, 0..Y-1
z / p	Current output-channel index	0..3
X / Y	Image width / height	positive
Z / P	Total channel count	4
i	Luminance from source RGB	approximately 0..255
u / v	Signed chroma components in native float mode	about -55..55 / -78..78
d	Direction from image center in 1/1024 turns	about -512..512
m	Distance from image center in pixels	0..M
D	One complete angular turn	1024
M	Half the full image diagonal	$\text{sqrt}(X^2+Y^2)/2$

2.2 Native float math vs. legacy integer math

Native Filter FabJS formulas use floating-point arithmetic. Fractions are preserved through intermediate calculations. This is a major difference from classic Filter Factory, where intermediate integer truncation was a defining limitation.

Native float mode: approximately 170

```
2/3*255
```

Historic .afs imports use legacy integer compatibility mode. In that mode, constants, variable reads, arithmetic results, and function results are truncated toward zero at key evaluation steps to better preserve old Filter Factory behavior.

Legacy integer mode: 0, because 2/3 truncates to 0

```
2/3*255
```

Do not deliberately write native filters around integer truncation unless you specifically want legacy behavior. Use floor(), ceil(), round(), or truncation-like constructions explicitly when quantization is part of the effect.

2.3 Formula syntax and parser limits

Each channel contains one expression. The language is expression-oriented, not JavaScript. You cannot declare variables, assign values, write loops, call arbitrary browser APIs, or use semicolon-separated statements.

- Whitespace is ignored.
- Identifiers are case-sensitive: X and x are different variables.
- Line comments beginning with // are allowed.
- Decimal and hexadecimal numeric literals are accepted.
- Scientific notation such as 1e-3 is not part of the current tokenizer; write 0.001 instead.
- A formula may contain nested function calls, arithmetic, comparisons, logical operators, ternary selection, and - on the CPU path - comma sequencing and stateful functions.

Limit	Maximum
Formula text	8,192 characters
Tokens	4,096
Syntax nodes	4,096
Nesting depth	128

```
clamp(c+20,0,255) // brighten the current channel
```

These limits are deliberate safety boundaries. Imported filters are subject to the same parser budgets.

2.4 Constants and variables

2.4.1 Numeric constants

```
42
-12.5
0.25
0xFF
```

Native formulas use finite JavaScript-style numeric values. Hexadecimal literals are useful for integer constants, but color-channel arithmetic is usually clearer in decimal.

2.4.2 Primary variables

Variable	Meaning
r, g, b, a	Source RGBA channels at the current pixel.
c	Current source channel selected by the output field.
i	Luminance: 0.299R + 0.587G + 0.114B.
u	Signed YUV-like chroma U; native nominal bounds are -55 to 55.
v	Signed YUV-like chroma V; native nominal bounds are -78 to 78.
x, y	Current pixel coordinates.
z, p	Current channel index, 0=R, 1=G, 2=B, 3=A.
d	Polar direction from image center. 1024 units = 360°.
m	Polar radius from image center in pixels.

2.4.3 Range and compatibility constants

Variable(s)	Value in native mode
R,G,B,A,C,I	255

U	110, the native U span
V	156, the native V span
rmax,gmax,bmax,amax,cmax,imax	255
umin / umax	-55 / 55
vmin / vmax	-78 / 78
dmin / dmax	-512 / 512
xmax / ymax	X / Y
mmax	M
rmin..zmin	0 except umin, vmin, dmin
t / tmin	0
tmax / total	1

Legacy .afs imports switch the chroma constant model to historic 0..255 U/V bounds. The still-image application keeps t at 0 and total at 1; those names exist for compatibility rather than animation.

2.5 Operators and precedence

Group	Operators	Notes
Unary	+ - ! ~	~ is CPU-only.
Multiply	* / %	Division/modulo by zero return 0.
Add	+ -	
Shift	<< >>	CPU-only.
Compare	< <= > >=	Returns 0 or 1.
Equality	== !=	Returns 0 or 1.
Bitwise	& ^ |	CPU-only; all share one precedence level.
Logical	&& ||	Short-circuit; both share one precedence level.
Ternary	condition ? yes : no	Expression selection.
Comma	,	CPU-only sequencing; lowest precedence.

Filter FabJS precedence is similar to C but not identical. In particular, && and || share the same precedence, and &, ^, and | also share one precedence. Use parentheses whenever mixed logical or bitwise expressions could be ambiguous.

Conditional inversion

```
i>128 ? 255-c : c
```

Logical and comparison operators return numeric 0 or 1. Any nonzero value is treated as true by conditions.

2.6 Controls and mappings

2.6.1 ctl(index)

ctl(i) reads one of the eight UI controls. Valid indices are 0 through 7. The raw control range is 0..255.

```
clamp(c+ctl(0)-128,0,255)
```

2.6.2 val(index, low, high)

val() maps the raw control continuously into a custom range. Unlike classic Filter Factory, native mode does not force the result to an integer.

```
val(0,-100,100)
```

low may be greater than high, which reverses the control direction.

2.6.3 map(index, value)

map(i,v) uses control pair i as a two-point remapping curve. Map 0 uses controls 0 and 1, map 1 uses controls 2 and 3, and so on. The input value is clamped to 0..255. Reversing the two controls reverses the output ramp.

```
map(0,i)
```

For most new filters, val() is easier to reason about. map() is primarily useful when you want an adjustable low/high threshold pair.

2.7 Image sampling

Function	Purpose
src(x,y,z)	Nearest-neighbor sample with edge clamping.
srcLinear(x,y,z)	Bilinear sample with edge clamping.
srcWrap(x,y,z)	Nearest sample with toroidal wraparound.
srcMirror(x,y,z)	Nearest sample with mirrored edge repetition.
rad(angle,distance,z)	Nearest polar sample around image center.

Sampling functions read the original source image, not partially rendered output. This prevents feedback loops and keeps deterministic formulas parallelizable.

Horizontal mirror

```
src(X-1-x,y,z)
```

Subpixel red-channel offset

```
srcLinear(x+val(0,-32,32),y+val(1,-32,32),0)
```

2.7.1 Edge behavior

src() and srcLinear() clamp coordinates to the nearest image edge. srcWrap() wraps coordinates across opposite edges. srcMirror() reflects coordinates back into the image. Choose the edge rule as part of the visual design: clamp produces smears at large offsets, wrap is ideal for seamless displacement, and mirror often hides borders in blur-like transforms.

2.7.2 Legacy aliases

src0(), src1(), rad0(), rad1(), cnv0(), and cnv1(), plus r0/r1-style variables, are currently compatibility aliases to the same single source image. Filter FabJS does not expose two simultaneous image sources through these names.

2.8 Numeric and shaping functions

Function	Meaning
min(a,b) / max(a,b)	Minimum / maximum.
abs(x)	Absolute value.
clamp(x,lo,hi)	Clamp x to lo..hi.
floor(x), ceil(x), round(x)	Explicit quantization.
fract(x)	Fractional part: x-floor(x).
sign(x)	-1, 0, or 1.

sqr(x)	Square x*x. Important: this is not square root.
sqrt(x)	Square root of max(0,x).
pow(x,y)	Power. CPU-only in direct formulas.
add(a,b,c)	min(a+b,c), a legacy helper.
sub(a,b,c)	max(abs(a-b),c), a legacy helper.
dif(a,b)	abs(a-b).
scl(v,inLo,inHi,outLo,outHi)	Linear range mapping; returns 0 if input range is zero.
mix(a,b,n,d)	a*n/d + b*(d-n)/d; returns 0 if d is zero.
lerp(a,b,t)	Linear interpolation. t can be 0..1 or 0..255.
step(edge,x)	0 when x<edge, otherwise 1.
smoothstep(lo,hi,x)	Smooth 0..1 Hermite transition.
bias(v,b) / gain(v,g)	Curve shaping for normalized values.

Contrast

```
clamp((c-128)*val(1,0.25,2.5)+128,0,255)
```

bias() and gain() clamp their first argument to 0..1. Their second parameter may be written as 0..1 or 0..255; values above 1 are interpreted on the 255 scale.

2.9 Polar coordinates

Filter FabJS uses 1024 angular units per full turn. Therefore 0 is to the right, 256 points downward, 512 points left, and -256 points upward. This convention matches screen coordinates where y increases downward.

Function / variable	Meaning
d	Angle of current pixel from image center.
m	Distance of current pixel from image center.
c2d(dx,dy)	Convert Cartesian offset to angle units.
c2m(dx,dy)	Convert Cartesian offset to radius.
r2x(angle,radius)	Horizontal offset for polar coordinate.
r2y(angle,radius)	Vertical offset for polar coordinate.
rad(angle,radius,z)	Sample the source at a polar coordinate around image center.
sin(a), cos(a)	Return amplitudes scaled to about +/-512.
tan(a)	Returns 1024*tan(angle); grows rapidly near quarter turns.

Polar twist/radial offset

```
rad(d+val(0,-64,64),m+val(1,-40,40),z)
```

When you need normal -1..1 trigonometric output, divide sin() or cos() by 512.

2.10 Procedural noise

Function	Return / use
hash2(x,y,seed)	Deterministic 0..1 hash. Integer-like coordinates are best for cells.
valueNoise(x,y,scale,seed)	Smooth value noise, 0..1.
perlin(x,y,scale,seed)	Gradient noise, normalized to 0..1.
worleyF1(x,y,scale,seed)	Nearest cellular feature distance, 0..1.
worleyF2(x,y,scale,seed)	Second-nearest cellular feature distance, 0..1.
fbm(x,y,scale,octaves,lacunarity,gain,seed)	Multi-octave Perlin fractal sum, normalized.
turbulence(x,y,scale,octaves,seed)	Absolute-value multi-octave noise.
ridged(x,y,scale,octaves,seed)	Squared ridge-style multi-octave noise.
periodicNoise(x,y,periodX,periodY,seed)	Tileable 0..1 value noise over requested periods.

Noise scale is measured in pixels. Larger scale values create broader features. FBM octaves are clamped to 1..12; lacunarity is forced above 1; gain is clamped to a stable 0.01..0.99 range.

Fractal clouds

```
fbm(x,y,val(0,12,180),5,2,0.5,val(1,1,9999))*255
```

Cellular edge field

```
(worleyF2(x,y,40,17)-worleyF1(x,y,40,17))*900
```

2.10.1 Stateful random: rnd() and rst()

rnd(a,b) returns an inclusive pseudo-random integer between the lower and upper endpoints. rst(seed) resets the sequential generator and returns 0. Because these functions depend on evaluation order, they are CPU-only and are not recommended for new GPU-oriented filters. Prefer hash2() and the stateless noise functions.

2.11 Gradients and patterns

Function	Meaning
wrap(v,size)	Positive modulo in the range 0..size.
mirror(v,size)	Ping-pong coordinate reflection.
linearGrad(x,y,x0,y0,x1,y1)	0..1 projection along a line segment, clamped.
radialGrad(x,y,cx,cy,radius)	1 at center, falling to 0 at radius.
angularGrad(x,y,cx,cy,offset)	Repeating 0..1 angle around center.
checker(x,y,width,height)	Alternating 0/1 checker cells.
brick(x,y,width,height,mortar,offset)	Brick-interior mask; 1 inside brick, 0 in mortar.
grid(x,y,cellWidth,cellHeight,lineWidth,feather)	Anti-aliased repeating grid-line mask.

```
checker(x,y,val(0,4,64),val(1,4,64))*255
```

```
linearGrad(x,y,0,0,X-1,Y-1)*255
```

```
radialGrad(x,y,X/2,Y/2,min(X,Y)/2)*255
```

angularGrad() accepts offset either as a normalized turn when |offset|<=1, or in Filter FabJS angle units when larger. brick() similarly treats |offset|<=1 as a fraction of brick width; larger values are interpreted as pixels.

2.12 Analytic shapes and masks

Analytic shape functions return normalized masks: 0 outside, 1 inside, with a smooth transition where feathering is nonzero. Coordinates, dimensions, widths, and feather values are in pixels. Negative dimensions are treated by absolute value.

Function	Signature / meaning
line	line(x,y,ax,ay,bx,by,width,feather)
circle	circle(x,y,cx,cy,radius,feather)
ring	ring(x,y,cx,cy,radius,width,feather)
box	box(x,y,cx,cy,width,height,rotation,feather)
triangle	triangle(x,y,ax,ay,bx,by,cx,cy,feather)
grid	grid(x,y,cellWidth,cellHeight,lineWidth,feather)
sierpinski	sierpinski(x,y,cx,cy,size,depth,feather)

box() rotation uses 1024 units per full turn. sierpinski() clamps depth to 0..10 and feathers both the outer edge and internal triangular holes.

```
circle(x,y,X/2,Y/2,min(X,Y)*0.25,val(0,0,4))*255
```

2.12.1 Combining masks

Use max() for a union, multiplication for an intersection, and subtraction followed by clamp() for a cutout.

```
max(circle(x,y,X/2,Y/2,80,1),box(x,y,X/2,Y/2,120,60,128,1))*255
```

```
clamp(circle(x,y,X/2,Y/2,80,1)-circle(x,y,X/2,Y/2,60,1),0,1)*255
```

2.12.2 Sierpiński gasket

```
sierpinski(x,y,X/2,Y/2,min(X,Y)*0.9,val(0,2,9),val(1,0,2))*255
```

This function produces a true finite-depth gasket mask rather than an approximate triangle pattern. For a colored fractal, multiply the mask by channel-specific foreground values or feed it into lerp().

2.13 Blend modes

Function	Definition
multiply(a,b[,opacity])	a*b/255
screen(a,b[,opacity])	255-(255-a)*(255-b)/255
overlay(a,b[,opacity])	Multiply below midtone, screen-like above.
softLight(a,b[,opacity])	Soft-light curve using normalized a and b.
difference(a,b[,opacity])	abs(a-b)

The optional opacity parameter may be 0..1 or 0..255. The blend functions clamp a and b to 0..255 before blending, then interpolate from the first input toward the blended value.

```
multiply(c,128,ctl(0))
```

For blending arbitrary formula values that are not intended as image channels, lerp() is usually more predictable.

2.14 Legacy and stateful functions

Feature	Meaning	Renderer
get(i)	Read one of 256 shared numeric cells.	CPU only
put(v,i)	Store v in a shared cell and return v.	CPU only
rnd(a,b)	Sequential pseudo-random integer.	CPU only
rst(seed)	Reset sequential random state; returns 0.	CPU only
pow(a,b)	Power function.	CPU only in direct formulas
~ & ^ | << >>	Bitwise and shift operators.	CPU only
comma operator	Evaluate left, then right; return right.	CPU only

The 256 get()/put() cells persist across pixels for the duration of a render and are cleared before each render. Their behavior therefore depends on raster evaluation order. They exist mainly for historic Filter Factory compatibility, not as a foundation for new parallel filters.

Classic names add(), sub(), dif(), mix(), scl(), cnv(), src0/src1 and related aliases are supported to improve .afs compatibility.

2.15 CPU and WebGPU compatibility

Native float formulas are WebGPU-compatible when they use only the stateless subset. The GPU subset includes source sampling, polar sampling, fixed 3x3 convolution, controls, numeric helpers, trigonometry, gradients, patterns, analytic masks, deterministic noise, ternary selection, comparisons, logical operators, and blend modes.

Language feature	WebGPU	CPU Worker
Arithmetic + - * / %	Yes	Yes
Comparisons / && / || / !	Yes	Yes
Ternary ? :	Yes	Yes
src/srcWrap/srcMirror/srcLinear/rad	Yes	Yes
cnv 3x3	Yes	Yes
Noise/gradients/patterns/shapes	Yes	Yes
Blend helpers	Yes	Yes
Legacy integer math	No	Yes
rnd/rst	No	Yes
get/put	No	Yes
Bitwise/shift/~	No	Yes
Comma sequencing	No	Yes
pow()	No	Yes

In Auto mode, unsupported operations do not make the filter invalid; they simply create a CPU fallback reason. Small floating-point differences between CPU and GPU are possible, especially around transcendental functions and threshold edges.

3. Tutorials

3.1 Pass through and invert

Pass through

```
R: r
G: g
B: b
A: a
```

Invert RGB, preserve alpha

```
R: 255-r
G: 255-g
B: 255-b
A: a
```

Preserving alpha with a is the safest default for effects that only alter color.

3.2 Luminance grayscale

```
R: i
G: i
B: i
```

```
A: a
```

i is computed once from source RGB using weighted luminance. Using i is clearer and less error-prone than manually repeating the coefficients.

3.3 Brightness and contrast

Brightness formula for R, G, and B

```
clamp(c+val(0,-100,100),0,255)
```

Contrast formula for R, G, and B

```
clamp((c-128)*val(1,0.25,2.5)+128,0,255)
```

The contrast formula expands or compresses distance from mid-gray. The final clamp prevents values from escaping the legal display range before later composition.

3.4 Duotone

```
R: scl(i,0,255,ctl(0),ctl(3))
G: scl(i,0,255,ctl(1),ctl(4))
B: scl(i,0,255,ctl(2),ctl(5))
A: a
```

Controls 0-2 define the shadow color; controls 3-5 define the highlight color. scl() maps luminance directly between those two colors.

3.5 Mirroring and offsets

Horizontal mirror

```
src(X-1-x,y,z)
```

Vertical mirror

```
src(x,Y-1-y,z)
```

Wrapped 2D offset

```
srcWrap(x+val(0,-X,X),y+val(1,-Y,Y),z)
```

The -1 in mirror formulas is essential. X is the width, but the final valid integer coordinate is X-1.

3.6 RGB channel split

```
R: srcLinear(x+val(0,-32,32),y+val(1,-32,32),0)
G: srcLinear(x+val(2,-32,32),y+val(3,-32,32),1)
B: srcLinear(x+val(4,-32,32),y+val(5,-32,32),2)
A: a
```

srcLinear() makes subpixel shifts smooth. Use srcWrap() instead when you prefer wraparound seams rather than edge clamping.

3.7 Mosaic

```
srcLinear(floor(x/val(0,2,64))*val(0,2,64)+val(0,2,64)/2,floor(y/val(1,2,64))*val(1,2,64)+val(1,2,64)/2,z)
```

The formula finds the center of the current block and samples that location. Apply it to all four channels if you want alpha mosaicked too; otherwise use a for alpha.

3.8 Procedural checker and gradients

Checkerboard

```
checker(x,y,val(0,4,64),val(1,4,64))*255
```

Diagonal gradient

```
linearGrad(x,y,0,0,X-1,Y-1)*255
```

Center-out radial gradient

```
radialGrad(x,y,X/2,Y/2,min(X,Y)/2)*255
```

These generators return normalized masks or fields, so multiply by 255 when you want a full grayscale image. When compositing, keep them in 0..1 form.

3.9 Fractal clouds

```
fbm(x,y,val(0,12,180),5,2,0.5,val(1,1,9999))*255
```

Control 0 changes feature size; control 1 is a deterministic seed. The fixed 5 octaves, lacunarity 2, and gain 0.5 are a balanced default. For a lower-cost effect, reduce octaves.

3.10 Noise displacement

```
srcLinear(x+(valueNoise(x,y,val(0,8,120),val(2,1,9999))-0.5)*val(1,0,52),y+
(valueNoise(x+431,y+719,val(0,8,120),val(2,1,9999))-0.5)*val(1,0,52),z)
```

Two decorrelated value-noise fields displace x and y. The constant offsets 431 and 719 prevent both axes from using the same noise sample. Since valueNoise() returns 0..1, subtracting 0.5 centers displacement around zero.

3.11 Analytic shape composition

A mask can be treated as an image-generation primitive or as a selector for another effect.

Union of a circle and rotated box

```
max(circle(x,y,X/2,Y/2,80,1),box(x,y,X/2,Y/2,120,60,128,1))*255
```

Paint white inside a feathered circle

```
lerp(c,255,circle(x,y,X/2,Y/2,80,2))
```

Because lerp() accepts 0..1 masks directly, shape functions compose naturally with image channels.

3.12 Sierpiński fractal

```
R: lerp(r,238*sierpinski(x,y,X/2,Y/2,min(X,Y)*0.9,val(0,2,9),val(1,0,2)),ctl(7))
G: lerp(g,232*sierpinski(x,y,X/2,Y/2,min(X,Y)*0.9,val(0,2,9),val(1,0,2)),ctl(7))
B: lerp(b,214*sierpinski(x,y,X/2,Y/2,min(X,Y)*0.9,val(0,2,9),val(1,0,2)),ctl(7))
A: a
```

The mask is GPU-compatible and finite-depth. Large depths increase visual frequency but are capped at 10 to keep the function bounded.

3.13 Convolution: blur and sharpen

3x3 box blur

```
cnv(1,1,1,1,1,1,1,1,1,9)
```

3x3 sharpen

```
cnv(0,-1,0,-1,5,-1,0,-1,0,1)
```

cnv() samples a fixed 3x3 neighborhood. Arguments 1-9 are row-major kernel weights from top-left to bottom-right; argument 10 is the divisor. A zero divisor returns 0. Edge sampling is clamped.

For multi-pass Gaussian blur, median filtering, or larger morphology, a single formula is not equivalent to a true pass graph. Prefer dedicated renderer operations when those become available rather than attempting large nested sampling approximations.

3.14 Blend helpers

```
multiply(c,128,ctl(0))
```

This darkens the current channel toward a 50% gray multiply result, with control 0 acting as blend opacity. Since the blend helper's first input is the base, opacity 0 returns c and opacity 255 returns the full multiply result.

3.15 Conditional threshold effects

```
i>128?255-c:c
```

Ternary expressions are compact and GPU-compatible. For soft transitions, smoothstep() generally produces better anti-aliasing than a hard comparison.

4. Debugging, safety, and performance

4.1 Common syntax errors

Problem	Bad	Correct
Function arity	abs(r,g)	abs(r)
Unknown helper	lerp(a,b,c,d)	lerp(a,b,t)
Missing ternary colon	i>128?255	i>128?255:0
Semicolon statement	c+20;	c+20
Assignment	x=20	Use a direct expression; assignments are

		not supported.
Scientific notation	1e-3	0.001
Wrong case	Min(r,g)	min(r,g)

4.2 Division and modulo by zero

The evaluator deliberately returns 0 for division or modulo by zero. This avoids exceptions but can hide bad control ranges. Protect denominators when zero would indicate an invalid parameter.

```
c/max(1,val(0,0,255))
```

4.3 Clamping and normalized fields

Image channels normally live in 0..255. Masks, gradients, hash/noise, and many shaping helpers use 0..1. Mixing these two scales is a common source of weak or blown-out effects.

- Multiply a 0..1 field by 255 only when converting it into a full channel image.
- Do not multiply a 0..1 mask by 255 before passing it as lerp()'s t unless you deliberately want 0..255 opacity semantics; both work, but 0..1 is easier to read.
- Clamp intermediate channel math when later operations assume 0..255.

4.4 CPU-only constructs

If a formula unexpectedly falls back to CPU, first look for rnd/rst, get/put, pow, bitwise operators, shifts, ~, comma sequencing, or legacy .afs math mode. Removing a single stateful construct can make an otherwise identical filter GPU-compatible.

4.5 Cost model

- Nearest sampling is cheaper than bilinear sampling.
- A single hash2() is cheaper than multi-octave noise.
- FBM, turbulence, and ridged cost roughly in proportion to octave count.
- Worley noise checks neighboring feature cells and is heavier than hash/value noise.
- cnv() always performs a 3x3 neighborhood convolution.
- Analytic shape functions are bounded, but deeply layered masks still add arithmetic cost.

Prefer a smaller number of well-chosen primitives over repeated duplicate subexpressions. The current typed IR may reuse unchanged programs across control-only renders, but it does not provide general user-defined local variables in formula text.

5. Import/export notes

Filter FabJS has a native JSON filter format and can import historic Filter Factory .afs files. Native v2 export is strictly normalized. Historic imports run in legacy integer math mode and are subject to the same formula length, token, node, and nesting limits as native formulas.

When translating an old Filter Factory formula manually, check the following before assuming visual parity:

- Classic integer arithmetic may need explicit floor/round behavior if converted to native float mode.
- Classic sqr() was often documented as square root; in current Filter FabJS sqr(x) means x*x and sqrt(x) is the square-root function.

- The original Filter Factory's random lookup behavior is not the recommended model; use Filter FabJS stateless noise for new work.
- Source 0/1 aliases do not create two independent image inputs in Filter FabJS.
- Filter FabJS always works with four RGBA channels internally.
- Classic preview-size workarounds are generally unnecessary because formulas are evaluated against the actual render dimensions.

6. Quick reference

Category	Functions
Sampling	src, srcLinear, srcWrap, srcMirror, rad, cnv
Controls	ctl, val, map
Basic math	min, max, abs, add, sub, dif, clamp, floor, ceil, round, fract, sign
Range/curves	mix, scl, lerp, step, smoothstep, bias, gain
Powers/trig	sqr, sqrt, pow, sin, cos, tan
Polar helpers	r2x, r2y, c2d, c2m
Noise	hash2, valueNoise, perlin, worleyF1, worleyF2, fbm, turbulence, ridged, periodicNoise
Coordinates/patterns	wrap, mirror, linearGrad, radialGrad, angularGrad, checker, brick
Masks	line, circle, ring, box, triangle, grid, sierpinski
Blends	multiply, screen, overlay, softLight, difference
Legacy/stateful	rnd, rst, get, put, src0, src1, rad0, rad1, cnv0, cnv1

6.1 Function signatures

Signature	Purpose
src(x,y,z)	sample, clamped nearest
srcLinear(x,y,z)	sample, clamped bilinear
srcWrap(x,y,z)	sample, wrapped nearest
srcMirror(x,y,z)	sample, mirrored nearest
rad(angle,radius,z)	polar sample
cnv(k00,k01,k02,k10,k11,k12,k20,k21,k22,div)	3x3 convolution
ctl(i)	raw control 0..255
val(i,a,b)	map control into a..b
map(i,v)	control-pair remapping
scl(v,a,b,c,d)	map v from a..b into c..d
lerp(a,b,t)	interpolate, t in 0..1 or 0..255
hash2(x,y,seed)	deterministic 0..1 hash
fbm(x,y,scale,octaves,lacunarity,gain,seed)	fractal Perlin sum
linearGrad(x,y,x0,y0,x1,y1)	0..1 linear gradient
radialGrad(x,y,cx,cy,r)	1..0 radial gradient
angularGrad(x,y,cx,cy,offset)	0..1 angular ramp
brick(x,y,w,h,mortar,offset)	brick interior mask
line(x,y,ax,ay,bx,by,width,feather)	line mask
circle(x,y,cx,cy,r,feather)	circle mask
ring(x,y,cx,cy,r,width,feather)	ring mask
box(x,y,cx,cy,w,h,rotation,feather)	rotated box mask
triangle(x,y,ax,ay,bx,by,cx,cy,feather)	triangle mask
grid(x,y,w,h,lineWidth,feather)	grid mask
sierpinski(x,y,cx,cy,size,depth,feather)	Sierpiński mask
multiply(a,b[,opacity])	blend
screen(a,b[,opacity])	blend
overlay(a,b[,opacity])	blend
softLight(a,b[,opacity])	blend

difference(a,b[,opacity])	blend

Appendix A. Formula-writing checklist

- Start with a pass-through filter and change one channel at a time.
- Decide whether a value is a channel value (usually 0..255) or normalized field/mask (usually 0..1).
- Use c when the same expression should apply independently to R, G, and B.
- Use explicit channel indices 0/1/2/3 when channels need different sampling.
- Use X-1 and Y-1 for final valid integer coordinates.
- Prefer val() over manual ctl()-128 arithmetic when a semantic range is known.
- Prefer hash2()/noise functions over rnd() for deterministic GPU-friendly randomness.
- Use srcLinear() for subpixel displacement; src()/srcWrap()/srcMirror() for crisp integer-like sampling.
- Use parentheses around mixed logical/bitwise expressions because precedence differs slightly from C.
- Preserve alpha with a unless the effect intentionally changes transparency.
- Check renderer status after adding stateful or legacy-only features.
- Keep formulas below parser budgets and avoid repeated giant subexpressions.

Appendix B. Corrections from classic Filter Factory assumptions

Several details in the historic Filter Factory literature should not be copied literally into Filter FabJS formulas:

Classic assumption	**Filter FabJS v2.4.6**
All arithmetic is integer.	Native filters use floating point; only legacy imports use integer-compatibility behavior.
sqr(x) means square root.	sqr(x) means x*x. Use sqrt(x) for square root.
Only nearest src() sampling exists.	srcLinear(), srcWrap(), and srcMirror() are native additions.
Randomness is mainly rnd().	Prefer deterministic hash/noise functions for GPU-compatible procedural work.
There are only old trigonometric/polar helpers.	Modern gradients, patterns, masks, and noise are built in.
get/put are a general optimization technique.	They are stateful CPU-only compatibility features; avoid them in new GPU-oriented filters.
Two-source src0/src1 names imply two inputs.	They are aliases to the same single source image in current Filter FabJS.
A filter should work around tiny preview dimensions.	Filter FabJS evaluates against the actual render dimensions; use X/Y for resolution-independent design, not preview correction.

The most important correction is sqr(): the old guide describes sqr(x) as square root, while current Filter FabJS deliberately implements sqr(x) as square and provides sqrt(x) separately. Treating these as interchangeable would produce severe visual errors.

Source basis

This guide was prepared from the current Filter FabJS v2.4.6 repository implementation, including the shipping formula parser, CPU evaluator, WGSL compiler, analytic-shape documentation, project-status documentation, built-in preset examples, and chroma contracts. The 1997 Filter Factory Programming Guide supplied with this request was used as a structural and historical reference, not copied as current syntax.